

MicroOS-Sim - Week 3 Testing

Testing Objective

The objective of Week 3 testing is to verify the new Memory, Stack, and FIFO Queue functionality added to the existing simulator. The testing also checks whether the new operations work correctly with the CPU and whether the expected results match the actual simulator results.

Testing Method

Each functionality is tested using a small input program or direct operation. The expected result is written before execution and the actual result is recorded after running the test in the simulator. A test is marked as passed only when the actual behaviour matches the expected behaviour.

1. Memory Testing

Test ID	Test Program	Expected Result	Actual Result	Status
MEM-01	Write the value 25 to memory address 10, then read address 10.	The value 25 should be returned from memory address 10.	The value 25 is returned from memory address 10.	PASS
MEM-02	Write values 15 and 30 to two different memory addresses and read both addresses.	Each address should return the value that was written to it.	Both addresses returned their corresponding values.	PASS
MEM-03	Reset the memory after storing a value.	The stored value should be cleared after the memory reset.	The stored value was cleared after the memory reset.	PASS

2. Stack Testing

Test ID	Test Program	Expected Result	Actual Result	Status
STK-01	PUSH 10, then PUSH 20.	The stack should contain 10 and 20, with 20 at the top.	The stack contained both values and 20 was at the top.	PASS
STK-02	PUSH 10, PUSH 20, then POP twice.	The first POP should return 20 and the second POP should return 10.	The first POP returned 20 and the second POP returned 10.	PASS
STK-03	Perform PUSH and POP operations while observing the Stack Pointer.	The Stack Pointer should change when values are pushed and popped.	The Stack Pointer changed correctly during the operations.	PASS
STK-04	Attempt to POP when the stack is empty.	The simulator should handle the empty stack condition without removing a value.	The empty stack condition was handled correctly.	PASS

3. FIFO Queue Testing

Test ID	Test Program	Expected Result	Actual Result	Status
QUE-01	ENQUEUE 10, ENQUEUE 20, ENQUEUE 30.	The queue should contain 10, 20, and 30 in that order.	The queue contained 10, 20, and 30 in that order.	PASS
QUE-02	ENQUEUE 10, ENQUEUE 20, ENQUEUE 30, then DEQUEUE three times.	The values should be removed as 10, 20, and 30.	The values were removed as 10, 20, and 30.	PASS
QUE-03	Fill the queue to its capacity.	The queue should report the full condition and should not accept another value.	The queue reported the full condition correctly.	PASS
QUE-04	Remove every value from the queue.	The queue should report the empty condition after the final dequeue.	The queue reported the empty condition correctly.	PASS

4. New Instruction Testing

Test ID	Instruction	Test Program	Expected Result	Actual Result	Status
INS-01	PUSH	PUSH 10 PUSH 20	20 should become the top value of the stack.	20 became the top value.	PASS
INS-02	POP	PUSH 10 PUSH 20 POP	The value 20 should be removed first.	The value 20 was removed first.	PASS
INS-03	ENQUEUE	ENQUEUE 10 ENQUEUE 20	10 should be at the front and 20 at the rear.	10 was at the front and 20 at the rear.	PASS
INS-04	DEQUEUE	ENQUEUE 10 ENQUEUE 20 DEQUEUE	The value 10 should be removed first.	The value 10 was removed first.	PASS

5. Queue Assembly Validation

The Queue implementation is validated through a processor-specific Assembly program. The program performs multiple Enqueue and Dequeue operations so that the FIFO order can be observed.

Test ID	Assembly Program	Expected Result	Actual Result	Status
ASM-01	ENQUEUE 10 ENQUEUE 20 ENQUEUE 30 DEQUEUE DEQUEUE DEQUEUE	The values should be removed in the order 10, 20, and 30.	The simulator removed the values in the order 10, 20, and 30.	PASS

Testing Summary

The Week 3 tests verify the required Memory Read and Write operations, Stack Pointer behaviour, PUSH and POP operations, FIFO Queue Enqueue and Dequeue operations, and the empty and full queue conditions. The Assembly validation confirms that values are removed from the queue in the same order in which they were added.